



## 06. PyTorch Transfer Learning Exercises

Welcome to the 06. PyTorch Transfer Learning exercise template notebook.

There are several questions in this notebook and it's your goal to answer them by writing Python and PyTorch code.

**Note:** There may be more than one solution to each of the exercises, don't worry too much about the *exact* right answer. Try to write some code that works first and then improve it if you can.

### Resources and solutions

- These exercises/solutions are based on [section 06. PyTorch Transfer Learning](#) of the Learn PyTorch for Deep Learning course by Zero to Mastery.

#### Solutions:

Try to complete the code below *before* looking at these.

- See a live [walkthrough of the solutions \(errors and all\) on YouTube](#).
- See an example [solutions notebook for these exercises on GitHub](#).

### 1. Make predictions on the entire test dataset and plot a confusion matrix for the results of our model compared to the truth labels.

- Note:** You will need to get the dataset and the trained model/retrain the model from notebook 06 to perform predictions.
- Check out [03. PyTorch Computer Vision section 10](#) for ideas.

```
# Import required libraries/code
import torch
import torchvision
import numpy as np
import matplotlib.pyplot as plt

from torch import nn
from torchvision import transforms, datasets

# Try to get torchinfo, install it if it doesn't work
try:
    from torchinfo import summary
except:
    print("[INFO] Couldn't find torchinfo... installing it.")
    !pip install -q torchinfo
    from torchinfo import summary

# Try to import the going_modular directory, download it from GitHub if it doesn't work
try:
    from going_modular.going_modular import data_setup, engine
except:
    # Get the going_modular scripts
    print("[INFO] Couldn't find going_modular scripts... downloading them from GitHub.")
    !git clone https://github.com/mrdbourke/pytorch-deep-learning
    !mv pytorch-deep-learning/going_modular .
    !rm -rf pytorch-deep-learning
    from going_modular.going_modular import data_setup, engine
```

```
# Setup device agnostic code
device = "cuda" if torch.cuda.is_available() else "cpu"
device
```

```
'cuda'
```

### Get data

```

import os
import requests
import zipfile

from pathlib import Path

# Setup path to data folder
data_path = Path("data/")
image_path = data_path / "pizza_steak_sushi"

# If the image folder doesn't exist, download it and prepare it...
if image_path.is_dir():
    print(f"{image_path} directory exists.")
else:
    print(f"Did not find {image_path} directory, creating one...")
    image_path.mkdir(parents=True, exist_ok=True)

# Download pizza, steak, sushi data
with open(data_path / "pizza_steak_sushi.zip", "wb") as f:
    request = requests.get("https://github.com/mrdbourke/pytorch-deep-learning/raw/main/data/pizza_steak_sushi.z
    print("Downloading pizza, steak, sushi data...")
    f.write(request.content)

# Unzip pizza, steak, sushi data
with zipfile.ZipFile(data_path / "pizza_steak_sushi.zip", "r") as zip_ref:
    print("Unzipping pizza, steak, sushi data...")
    zip_ref.extractall(image_path)

# Remove .zip file
os.remove(data_path / "pizza_steak_sushi.zip")

# Setup Dirs
train_dir = image_path / "train"
test_dir = image_path / "test"

data/pizza_steak_sushi directory exists.

```

## ▼ Prepare data

```

# Create a transforms pipeline
simple_transform = transforms.Compose([
    transforms.Resize((224, 224)), # 1. Reshape all images to 224x224 (though some models may require different size
    transforms.ToTensor(), # 2. Turn image values to between 0 & 1
    transforms.Normalize(mean=[0.485, 0.456, 0.406], # 3. A mean of [0.485, 0.456, 0.406] (across each colour channe
    std=[0.229, 0.224, 0.225]) # 4. A standard deviation of [0.229, 0.224, 0.225] (across each
])

```

```

# Create training and testing DataLoader's as well as get a list of class names
train_dataloader, test_dataloader, class_names = data_setup.create_data_loaders(train_dir=train_dir,
                                                                                test_dir=test_dir,
                                                                                transform=simple_transform, # resize,
                                                                                batch_size=32) # set mini-batch size

train_dataloader, test_dataloader, class_names

(<torch.utils.data.dataloader.DataLoader at 0x78a4333399a0>,
 <torch.utils.data.dataloader.DataLoader at 0x78a335ae4560>,
 ['pizza', 'steak', 'sushi'])

```

## ▼ Get and prepare a pretrained model

```

# Setup the model with pretrained weights and send it to the target device
model_0 = torchvision.models.efficientnet_b0(pretrained=True).to(device)
#model_0 # uncomment to output (it's very long)

/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is
warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight
warnings.warn(msg)

# Freeze all base layers in the "features" section of the model (the feature extractor) by setting requires_grad=Fal
for param in model_0.features.parameters():

```

```
param.requires_grad = False
```

```
# Set the manual seeds
torch.manual_seed(42)
torch.cuda.manual_seed(42)

# Get the length of class_names (one output unit for each class)
output_shape = len(class_names)

# Recreate the classifier layer and seed it to the target device
model_0.classifier = torch.nn.Sequential(
    torch.nn.Dropout(p=0.2, inplace=True),
    torch.nn.Linear(in_features=1280,
                    out_features=output_shape, # same number of output units as our number of classes
                    bias=True)).to(device)
```

## ✓ Train model

```
# Define loss and optimizer
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model_0.parameters(), lr=0.001)
```

```
# Set the random seeds
torch.manual_seed(42)
torch.cuda.manual_seed(42)

# Start the timer
from timeit import default_timer as timer
start_time = timer()

# Setup training and save the results
model_0_results = engine.train(model=model_0,
                               train_dataloader=train_dataloader,
                               test_dataloader=test_dataloader,
                               optimizer=optimizer,
                               loss_fn=loss_fn,
                               epochs=5,
                               device=device)

# End the timer and print out how long it took
end_time = timer()
print(f"[INFO] Total training time: {end_time-start_time:.3f} seconds")
```

```
100%          5/5 [00:10<00:00, 2.10s/it]
Epoch: 1 | train_loss: 1.0895 | train_acc: 0.4414 | test_loss: 0.9202 | test_acc: 0.5085
Epoch: 2 | train_loss: 0.8682 | train_acc: 0.7734 | test_loss: 0.8022 | test_acc: 0.7434
Epoch: 3 | train_loss: 0.7771 | train_acc: 0.7812 | test_loss: 0.7399 | test_acc: 0.7737
Epoch: 4 | train_loss: 0.7249 | train_acc: 0.7422 | test_loss: 0.6472 | test_acc: 0.8864
Epoch: 5 | train_loss: 0.6445 | train_acc: 0.7812 | test_loss: 0.6244 | test_acc: 0.8968
[INFO] Total training time: 10.579 seconds
```

## ✓ Make predictions on the entire test dataset with the model

```
len(test_dataloader)
```

```
3
```

```
from tqdm.auto import tqdm

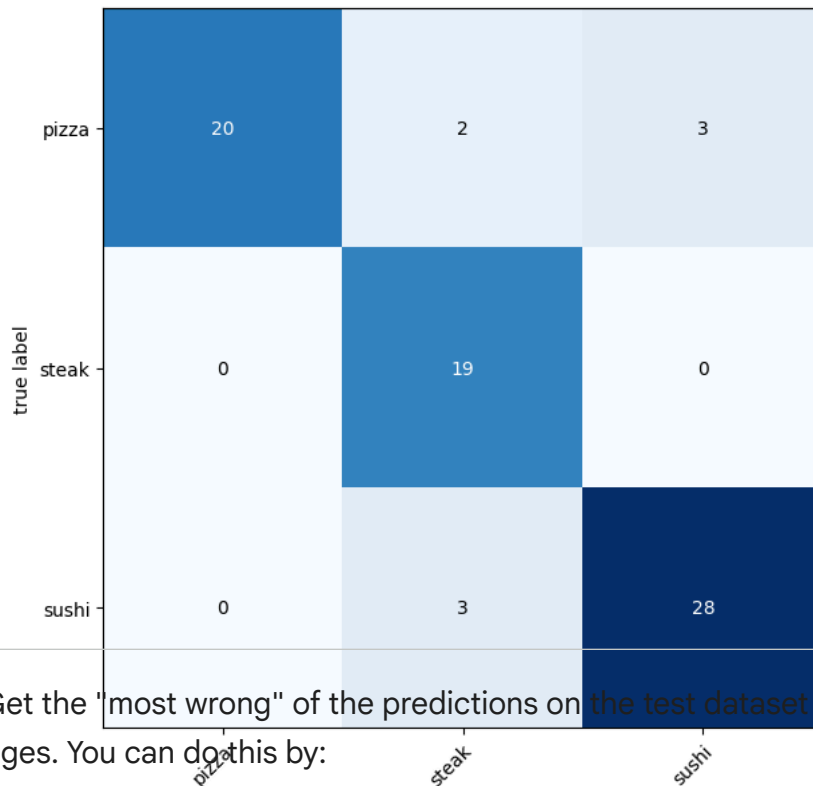
test_preds = []
model_0.eval()

with torch.inference_mode():
    for X, y in tqdm(test_dataloader):
        X, y = X.to(device), y.to(device)

        # pass data through the model and get logits
        test_logits = model_0(X)

        # convert pred logits to pred probs
        pred_probs = torch.softmax(test_logits, dim=1)
```





2. Get the "most wrong" of the predictions on the test dataset and plot the 5 "most wrong" images. You can do this by:

- Predicting across all of the test dataset, storing the labels and predicted probabilities.
- Sort the predictions by *wrong prediction* and then *descending predicted probabilities*, this will give you the wrong predictions with the *highest* prediction probabilities, in other words, the "most wrong".
- Plot the top 5 "most wrong" images, why do you think the model got these wrong?

You'll want to:

- Create a DataFrame with sample, label, prediction, pred prob
- Sort DataFrame by correct (does label == prediction)
- Sort DataFrame by pred prob (descending)
- Plot the top 5 "most wrong" image predictions

```
# Get all test data paths
from pathlib import Path
test_data_paths = list(Path(test_dir).glob("*/*.jpg"))
test_labels = [path.parent.stem for path in test_data_paths]

# Create a function to return a list of dictionaries with sample, label, prediction, pred prob
def pred_and_store(test_paths, model, transform, class_names, device):
    test_pred_list = []
    for path in tqdm(test_paths):
        # Create empty dict to store info for each sample
        pred_dict = {}

        # Get sample path
        pred_dict["image_path"] = path

        # Get class name
        class_name = path.parent.stem
        pred_dict["class_name"] = class_name

        # Get prediction and prediction probability
        from PIL import Image
        img = Image.open(path) # open image
        transformed_image = transform(img).unsqueeze(0) # transform image and add batch dimension
        model.eval()
        with torch.inference_mode():
            pred_logit = model(transformed_image.to(device))
            pred_prob = torch.softmax(pred_logit, dim=1)
            pred_label = torch.argmax(pred_prob, dim=1)
            pred_class = class_names[pred_label.cpu()]

        # Make sure things in the dictionary are back on the CPU
```

```

# Make sure things in the dictionary are back on the cpu
pred_dict["pred_prob"] = pred_prob.unsqueeze(0).max().cpu().item()
pred_dict["pred_class"] = pred_class

# Does the pred match the true label?
pred_dict["correct"] = class_name == pred_class

# print(pred_dict)
# Add the dictionary to the list of preds
test_pred_list.append(pred_dict)

return test_pred_list

test_pred_dicts = pred_and_store(test_paths=test_data_paths,
                                model=model_0,
                                transform=simple_transform,
                                class_names=class_names,
                                device=device)

test_pred_dicts[:5]

```

```

100% 75/75 [00:02<00:00, 69.21it/s]
[{'image_path': PosixPath('data/pizza_steak_sushi/test/steak/354513.jpg'),
  'class_name': 'steak',
  'pred_prob': 0.5341949462890625,
  'pred_class': 'steak',
  'correct': True},
 {'image_path': PosixPath('data/pizza_steak_sushi/test/steak/1302563.jpg'),
  'class_name': 'steak',
  'pred_prob': 0.8452677726745605,
  'pred_class': 'steak',
  'correct': True},
 {'image_path': PosixPath('data/pizza_steak_sushi/test/steak/690177.jpg'),
  'class_name': 'steak',
  'pred_prob': 0.7824719548225403,
  'pred_class': 'steak',
  'correct': True},
 {'image_path': PosixPath('data/pizza_steak_sushi/test/steak/1285886.jpg'),
  'class_name': 'steak',
  'pred_prob': 0.46011871099472046,
  'pred_class': 'steak',
  'correct': True},
 {'image_path': PosixPath('data/pizza_steak_sushi/test/steak/673127.jpg'),
  'class_name': 'steak',
  'pred_prob': 0.7858447432518005,
  'pred_class': 'steak',
  'correct': True}]

```

```

# Turn the test_pred_dicts into a DataFrame
import pandas as pd
test_pred_df = pd.DataFrame(test_pred_dicts)
# Sort DataFrame by correct then by pred_prob
top_5_most_wrong = test_pred_df.sort_values(by=["correct", "pred_prob"], ascending=[True, False]).head()
top_5_most_wrong

```

|    | image_path                                    | class_name | pred_prob | pred_class | correct |                                                                                       |
|----|-----------------------------------------------|------------|-----------|------------|---------|---------------------------------------------------------------------------------------|
| 21 | data/pizza_steak_sushi/test/sushi/684266.jpg  | sushi      | 0.546966  | steak      | False   |  |
| 35 | data/pizza_steak_sushi/test/sushi/1172255.jpg | sushi      | 0.449595  | steak      | False   |  |
| 65 | data/pizza_steak_sushi/test/pizza/971934.jpg  | pizza      | 0.444701  | sushi      | False   |  |
| 72 | data/pizza_steak_sushi/test/pizza/2508636.jpg | pizza      | 0.435138  | sushi      | False   |                                                                                       |
| 51 | data/pizza_steak_sushi/test/pizza/309892.jpg  | pizza      | 0.426874  | sushi      | False   |                                                                                       |

Next steps: [Generate code with top\\_5\\_most\\_wrong](#) [New interactive sheet](#)

```

import torchvision
import matplotlib.pyplot as plt
# Plot the top 5 most wrong images
for row in top_5_most_wrong.iterrows():
    row = row[1]
    image_path = row[0]
    true_label = row[1]
    pred_prob = row[2]
    pred_class = row[3]

```

```
# Plot the image and various details
img = torchvision.io.read_image(str(image_path)) # get image as tensor
plt.figure()
plt.imshow(img.permute(1, 2, 0)) # matplotlib likes images in [height, width, color_channels]
plt.title(f"True: {true_label} | Pred: {pred_class} | Prob: {pred_prob:.3f}")
plt.axis(False);
```

3. Predict on your own image of pizza/steak/sushi - how does the model go? What happens if you predict on an image that isn't pizza/steak/sushi?

- Here you can get an image from a website like <http://www.unsplash.com> to try it out or you can upload your own.

```
# Get an image of pizza/steak/sushi
!wget https://images.unsplash.com/photo-1588315029754-2dd089d39a1a
!cp photo-1588315029754-2dd089d39a1a pizza.jpg

from PIL import Image

with Image.open("pizza.jpg") as img:
    # Reduce the image size and width by 10
    (width, height) = (img.width // 10, img.height // 10)
    img_resized = img.resize((width, height))

img_resized
```



True: sushi | Pred: steak | Prob: 0.547  
2025-12-30 22:09:15 (198 MB/s) - photo-1588315029754-2dd089d39a1a' saved [3208960/3208960]



```
# Make a function to pred and plot images
def pred_and_plot(image_path, model, transform, class_names, device=device):
    # open image
    image = Image.open(image_path)

    # transform image
    transformed_image = transform(image)

    # pred on image
    model.eval()
    with torch.inference_mode():
        pred_logits = model(transformed_image.unsqueeze(0).to(device))
        pred_label = torch.argmax(torch.softmax(pred_logits, dim=1), dim=1)

    # plot image and pred
    plt.figure()
    plt.imshow(image)
    plt.title(f"Pred: {class_names[pred_label]}")
    plt.axis(False);

pred_and_plot(image_path="pizza.jpg",
               model=model_0,
               transform=simple_transform,
               class_names=class_names)
```

A whole pizza with a thick crust, topped with melted cheese, mushrooms, red onions, and cherry tomatoes. It is served on a wooden board with a fork and knife nearby.

[https://colab.research.google.com/drive/1hj5PaDRxD49QMRsbT2Xo46OKehN1EqYP#scrollTo=qSpRiJRIU0q\\_&printMode=true](https://colab.research.google.com/drive/1hj5PaDRxD49QMRsbT2Xo46OKehN1EqYP#scrollTo=qSpRiJRIU0q_&printMode=true)

```
# Try again on a photo of steak from unsplash.com
!wget https://images.unsplash.com/photo-1546964124-0cce460f38ef
!cp photo-1546964124-0cce460f38ef steak.jpg
```

```
pred_and_plot("steak.jpg",
              model=model_0,
              transform=simple_transform,
              class_names=class_names)
```

```
- 2025-12-30 22:10:07-- https://images.unsplash.com/photo-1546964124-0cce460f38ef
Resolving images.unsplash.com (images.unsplash.com)... 151.101.2.208, 151.101.66.208, 151.101.130.208, ...
Connecting to images.unsplash.com (images.unsplash.com)|151.101.2.208|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3617320 (3.4M) [image/jpeg]
Saving to: 'photo-1546964124-0cce460f38ef'
```

```
photo-1546964124-0c 100%[=====] 3.45M --.-KB/s in 0.008s
```

```
2025-12-30 22:10:07. (449 MB/s) - 'photo-1546964124-0cce460f38ef' saved [3617320/3617320]
true: pizza | Pred: sushi | Prob: 0.427
```

Pred: steak



```
# Get an image of not pizza/steak/sushi
!wget https://images.unsplash.com/photo-1570913149827-d2ac84ab3f9a
!cp photo-1570913149827-d2ac84ab3f9a apple.jpg
```

```
pred_and_plot(image_path="apple.jpg",
              model=model_0,
              transform=simple_transform,
              class_names=class_names)
```

--2025-12-30 22:10:16-- <https://images.unsplash.com/photo-1570913149827-d2ac84ab3f9a>

Resolving images.unsplash.com (images.unsplash.com)... 151.101.2.208, 151.101.66.208, 151.101.130.208, ...

Connecting to images.unsplash.com (images.unsplash.com)|151.101.2.208|:443... connected.

HTTP request sent, awaiting response... 200 OK

Length: 2006529 (1.9 MiB) [image/png]

Saving to: 'photo-1570913149827-d2ac84ab3f9a'

photo-1570913149827 100%[=====] 1.91M --.-KB/s in 0.007s

- See the model in notebook 06 part 3 for reference: [https://www.learnpytorch.io/06\\_pytorch\\_transfer\\_learning/#3-getting-a-pretrained-model](https://www.learnpytorch.io/06_pytorch_transfer_learning/#3-getting-a-pretrained-model)

Pred: sushi

```
# Recreate a new model
import torchvision
from torch import nn
model_1 = torchvision.models.efficientnet_b0(pretrained=True).to(device)
```

```
# Freeze the base layers
for param in model_1.features.parameters():
    param.requires_grad = False
```

```
# Change the classification head
torch.manual_seed(42)
model_1.classifier = torch.nn.Sequential(
    nn.Dropout(p=0.2, inplace=True),
    nn.Linear(in_features=1280, out_features=len(class_names), bias=True)
).to(device)
```

```
summary(model_1,
        input_size=[32, 3, 224, 224],
        col_names=["input_size", "output_size", "trainable"])
```

```
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is
warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight
warnings.warn(msg)
```

| Layer (type:depth-idx)     | Input Shape        | Output Shape       | Trainable |
|----------------------------|--------------------|--------------------|-----------|
| EfficientNet               | [32, 3, 224, 224]  | [32, 3]            | Partial   |
| └Sequential: 1-1           | [32, 3, 224, 224]  | [32, 1280, 7, 7]   | False     |
| └Conv2dNormActivation: 2-1 | [32, 3, 224, 224]  | [32, 32, 112, 112] | False     |
| └Conv2d: 3-1               | [32, 3, 224, 224]  | [32, 32, 112, 112] | False     |
| └BatchNorm2d: 3-2          | [32, 32, 112, 112] | [32, 32, 112, 112] | False     |
| └SiLU: 3-3                 | [32, 32, 112, 112] | [32, 32, 112, 112] | --        |
| └Sequential: 2-2           | [32, 32, 112, 112] | [32, 16, 112, 112] | False     |
| └MBConv: 3-4               | [32, 32, 112, 112] | [32, 16, 112, 112] | False     |
| └Sequential: 2-3           | [32, 16, 112, 112] | [32, 24, 56, 56]   | False     |
| └MBConv: 3-5               | [32, 16, 112, 112] | [32, 24, 56, 56]   | False     |
| └MBConv: 3-6               | [32, 24, 56, 56]   | [32, 24, 56, 56]   | False     |
| └Sequential: 2-4           | [32, 24, 56, 56]   | [32, 40, 28, 28]   | False     |
| └MBConv: 3-7               | [32, 24, 56, 56]   | [32, 40, 28, 28]   | False     |
| └MBConv: 3-8               | [32, 40, 28, 28]   | [32, 40, 28, 28]   | False     |
| └Sequential: 2-5           | [32, 40, 28, 28]   | [32, 80, 14, 14]   | False     |
| └MBConv: 3-9               | [32, 40, 28, 28]   | [32, 80, 14, 14]   | False     |
| └MBConv: 3-10              | [32, 80, 14, 14]   | [32, 80, 14, 14]   | False     |
| └MBConv: 3-11              | [32, 80, 14, 14]   | [32, 80, 14, 14]   | False     |
| └Sequential: 2-6           | [32, 80, 14, 14]   | [32, 112, 14, 14]  | False     |
| └MBConv: 3-12              | [32, 80, 14, 14]   | [32, 112, 14, 14]  | False     |
| └MBConv: 3-13              | [32, 112, 14, 14]  | [32, 112, 14, 14]  | False     |
| └MBConv: 3-14              | [32, 112, 14, 14]  | [32, 112, 14, 14]  | False     |
| └Sequential: 2-7           | [32, 112, 14, 14]  | [32, 192, 7, 7]    | False     |
| └MBConv: 3-15              | [32, 112, 14, 14]  | [32, 192, 7, 7]    | False     |
| └MBConv: 3-16              | [32, 192, 7, 7]    | [32, 192, 7, 7]    | False     |
| └MBConv: 3-17              | [32, 192, 7, 7]    | [32, 192, 7, 7]    | False     |
| └MBConv: 3-18              | [32, 192, 7, 7]    | [32, 192, 7, 7]    | False     |
| └Sequential: 2-8           | [32, 192, 7, 7]    | [32, 320, 7, 7]    | False     |
| └MBConv: 3-19              | [32, 192, 7, 7]    | [32, 320, 7, 7]    | False     |
| └Conv2dNormActivation: 2-9 | [32, 320, 7, 7]    | [32, 1280, 7, 7]   | False     |
| └Conv2d: 3-20              | [32, 320, 7, 7]    | [32, 1280, 7, 7]   | False     |
| └BatchNorm2d: 3-21         | [32, 1280, 7, 7]   | [32, 1280, 7, 7]   | False     |
| └SiLU: 3-22                | [32, 1280, 7, 7]   | [32, 1280, 7, 7]   | --        |
| └AdaptiveAvgPool2d: 1-2    | [32, 1280, 7, 7]   | [32, 1280, 1, 1]   | --        |
| └Sequential: 1-3           | [32, 1280]         | [32, 3]            | True      |
| └Dropout: 2-10             | [32, 1280]         | [32, 1280]         | --        |
| └Linear: 2-11              | [32, 1280]         | [32, 3]            | True      |

Total params: 4,011,391

Trainable params: 3,843

Non-trainable params: 4,007,548

```
Total mult-adds (Units.GIGABYTES): 12.31
```

```
=====
Input size (MB): 19.27
Forward/backward pass size (MB): 3452.09
Params size (MB): 16.05
Estimated Total Size (MB): 3487.41
=====
```

```
# Set the random seeds
torch.manual_seed(42)
torch.cuda.manual_seed(42)

# Start the timer
from timeit import default_timer as timer
start_time = timer()

# Create loss and optimizer
loss_fn = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model_1.parameters(), lr=0.001)

# Setup training and save the results
model_1_results = engine.train(model=model_1,
                               train_dataloader=train_dataloader,
                               test_dataloader=test_dataloader,
                               optimizer=optimizer,
                               loss_fn=loss_fn,
                               epochs=10,
                               device=device)

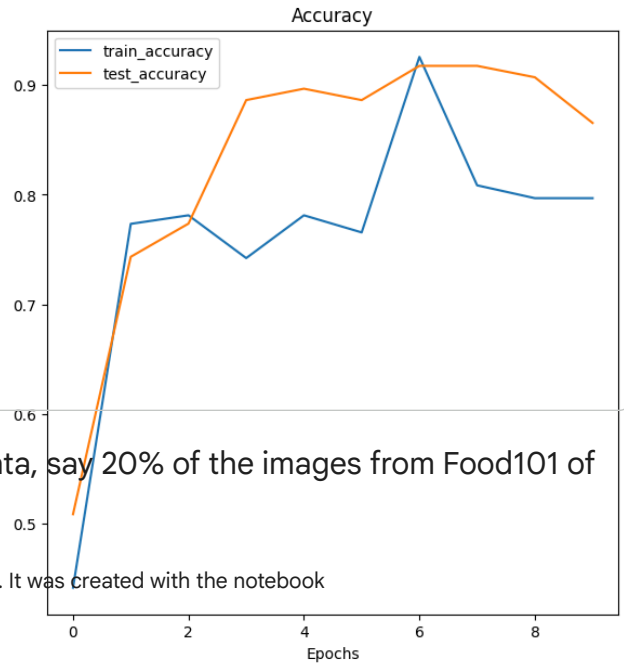
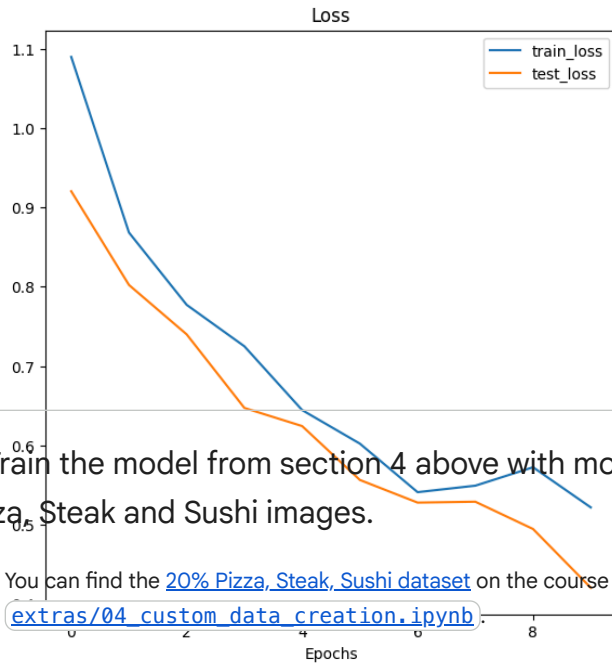
# End the timer and print out how long it took
end_time = timer()
print(f"[INFO] Total training time: {end_time-start_time:.3f} seconds")
```

```
100% 10/10 [00:18<00:00, 1.67s/it]
Epoch: 1 | train_loss: 1.0895 | train_acc: 0.4414 | test_loss: 0.9202 | test_acc: 0.5085
Epoch: 2 | train_loss: 0.8682 | train_acc: 0.7734 | test_loss: 0.8022 | test_acc: 0.7434
Epoch: 3 | train_loss: 0.7771 | train_acc: 0.7812 | test_loss: 0.7399 | test_acc: 0.7737
Epoch: 4 | train_loss: 0.7249 | train_acc: 0.7422 | test_loss: 0.6472 | test_acc: 0.8864
Epoch: 5 | train_loss: 0.6445 | train_acc: 0.7812 | test_loss: 0.6244 | test_acc: 0.8968
Epoch: 6 | train_loss: 0.6022 | train_acc: 0.7656 | test_loss: 0.5565 | test_acc: 0.8864
Epoch: 7 | train_loss: 0.5411 | train_acc: 0.9258 | test_loss: 0.5279 | test_acc: 0.9176
Epoch: 8 | train_loss: 0.5493 | train_acc: 0.8086 | test_loss: 0.5291 | test_acc: 0.9176
Epoch: 9 | train_loss: 0.5725 | train_acc: 0.7969 | test_loss: 0.4947 | test_acc: 0.9072
Epoch: 10 | train_loss: 0.5219 | train_acc: 0.7969 | test_loss: 0.4207 | test_acc: 0.8655
[INFO] Total training time: 18.021 seconds
```

```
# Get the plot_loss_curves() function from helper_functions.py, download the file if we don't have it
try:
    from helper_functions import plot_loss_curves
except:
    print("[INFO] Couldn't find helper_functions.py, downloading...")
    with open("helper_functions.py", "wb") as f:
        import requests
        request = requests.get("https://raw.githubusercontent.com/mrdbourke/pytorch-deep-learning/main/helper_functions.py")
        f.write(request.content)
    from helper_functions import plot_loss_curves

# Plot the loss curves of our model
plot_loss_curves(model_1_results)
```

[INFO] Couldn't find helper\_functions.py, downloading...



5. Train the model from section 4 above with more data, say 20% of the images from Food101 of Pizza, Steak and Sushi images.

- You can find the [20% Pizza, Steak, Sushi dataset](#) on the course GitHub. It was created with the notebook

[extras/04\\_custom\\_data\\_creation.ipynb](#)

#### Get 20% data

```
import os
import requests
import zipfile

from pathlib import Path

# Setup path to data folder
data_path = Path("data/")
image_path = data_path / "pizza_steak_sushi_20_percent"
image_data_zip_path = "pizza_steak_sushi_20_percent.zip"

# If the image folder doesn't exist, download it and prepare it...
if image_path.is_dir():
    print(f"{image_path} directory exists.")
else:
    print(f"Did not find {image_path} directory, creating one...")
    image_path.mkdir(parents=True, exist_ok=True)

# Download pizza, steak, sushi data
with open(data_path / image_data_zip_path, "wb") as f:
    request = requests.get("https://github.com/mrdbourke/pytorch-deep-learning/raw/main/data/pizza_steak_sushi_20_percent.zip")
    print("Downloading pizza, steak, sushi data...")
    f.write(request.content)

# Unzip pizza, steak, sushi data
with zipfile.ZipFile(data_path / image_data_zip_path, "r") as zip_ref:
    print("Unzipping pizza, steak, sushi 20% data...")
    zip_ref.extractall(image_path)

# Remove .zip file
os.remove(data_path / image_data_zip_path)

# Setup Dirs
train_dir_20_percent = image_path / "train"
test_dir_20_percent = image_path / "test"

train_dir_20_percent, test_dir_20_percent

data/pizza_steak_sushi_20_percent directory exists.
(PosixPath('data/pizza_steak_sushi_20_percent/train'),
 PosixPath('data/pizza_steak_sushi_20_percent/test'))
```

#### Create DataLoaders

```
# Create a transforms pipeline
simple_transform = transforms.Compose([
    transforms.Resize((224, 224)), # 1. Reshape all images to 224x224 (though some models may require different size
    transforms.ToTensor(), # 2. Turn image values to between 0 & 1
    transforms.Normalize(mean=[0.485, 0.456, 0.406], # 3. A mean of [0.485, 0.456, 0.406] (across each colour channel
                        std=[0.229, 0.224, 0.225]) # 4. A standard deviation of [0.229, 0.224, 0.225] (across each
])
```

```
# Create training and testing DataLoader's as well as get a list of class names
train_dataloader_20_percent, test_dataloader_20_percent, class_names = data_setup.create_data_loaders(train_dir=train_dir,
                                                                                                     test_dir=test_dir,
                                                                                                     transform=simple_transform,
                                                                                                     batch_size=32)

train_dataloader_20_percent, test_dataloader_20_percent, class_names
```

```
(<torch.utils.data.dataloader.DataLoader at 0x78a26b4239e0>,
<torch.utils.data.dataloader.DataLoader at 0x78a26b4228d0>,
['pizza', 'steak', 'sushi'])
```

## Get a pretrained model

```
# Create a new model for 20 percent of the data
model_2 = torchvision.models.efficientnet_b0(pretrained=True).to(device)

# Freeze all the base layers
for param in model_2.features.parameters():
    param.requires_grad = False

# Change the classifier head
torch.manual_seed(42)
model_2.classifier = nn.Sequential(
    nn.Dropout(p=0.2, inplace=True),
    nn.Linear(in_features=1280, out_features=3, bias=True)
).to(device)
```

## Train a model with 20% of the data

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model_2.parameters(), lr=0.001)

# Set the random seeds
torch.manual_seed(42)
torch.cuda.manual_seed(42)

# Start the timer
from timeit import default_timer as timer
start_time = timer()

# Setup training and save the results
model_2_results = engine.train(model=model_2,
                               train_dataloader=train_dataloader_20_percent,
                               test_dataloader=test_dataloader_20_percent,
                               optimizer=optimizer,
                               loss_fn=loss_fn,
                               epochs=5,
                               device=device)

# End the timer and print out how long it took
end_time = timer()
print(f"[INFO] Total training time: {end_time-start_time:.3f} seconds")
```

```
100% 5/5 [00:18<00:00, 3.60s/it]
Epoch: 1 | train_loss: 0.9747 | train_acc: 0.5667 | test_loss: 0.6472 | test_acc: 0.8949
Epoch: 2 | train_loss: 0.7067 | train_acc: 0.8167 | test_loss: 0.5291 | test_acc: 0.9006
Epoch: 3 | train_loss: 0.5866 | train_acc: 0.8271 | test_loss: 0.4519 | test_acc: 0.8881
Epoch: 4 | train_loss: 0.4555 | train_acc: 0.8917 | test_loss: 0.4089 | test_acc: 0.8881
Epoch: 5 | train_loss: 0.4520 | train_acc: 0.8896 | test_loss: 0.3788 | test_acc: 0.9068
[INFO] Total training time: 18.170 seconds
```



```
# Check results with 10% of data for 5 epochs
max(model_0_results["test_acc"]), min(model_0_results["test_loss"])
```

```
(0.89678030303031, 0.6243551969528198)
```

```
# Check results with 20% of data for 5 epochs
max(model_2_results["test_acc"]), min(model_2_results["test_loss"])
```

```
(0.90681818181819, 0.3788062870502472)
```

Start coding or [generate](#) with AI.

6. Try a different model from [torchvision.models](#) on the Pizza, Steak, Sushi data, how does this model perform?

- You'll have to change the size of the classifier layer to suit our problem.
- You may want to try an EfficientNet with a higher number than our B0, perhaps `torchvision.models.efficientnet_b2()`?
  - Note:** Depending on the model you use you will have to prepare/transform the data in a certain way.

```
# Create a transform to transform the data
from torchvision import transforms, models
effnet_b2_transform = transforms.Compose([
    transforms.Resize((288, 288)), # effnet_b2 takes images of size 288, 288 - https://github.com/pytorch/vision/blob/
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])
```

```
# Use 10% data sample for effnet_b2 to compare to model_0_results
train_dir, test_dir
```

```
(PosixPath('data/pizza_steak_sushi/train'),
 PosixPath('data/pizza_steak_sushi/test'))
```

```
# Import the data
train_dataloader_effnet_b2, test_dataloader_effnet_b2, class_names = data_setup.create_data_loaders(train_dir=train_dir,
                                                                                                     test_dir=test_dir,
                                                                                                     transform=effnet_b2_transform,
                                                                                                     batch_size=32)
```

```
# Create a effnet_b2 new model - https://pytorch.org/vision/stable/generated/torchvision.models.efficientnet_b2.html
model_3 = torchvision.models.efficientnet_b2(pretrained=True).to(device)
```

```
# Freeze the base layers
for param in model_3.parameters():
    param.requires_grad = False
```

```
# Change the classifier head (to suit our problem)
torch.manual_seed(42)
model_3.classifier = nn.Sequential(
    nn.Dropout(p=0.3, inplace=True),
    nn.Linear(in_features=1408, out_features=len(class_names), bias=True)
).to(device)
```

```
# Get a summary of the new model
summary(model_3,
        input_size=[32, 3, 288, 288],
        col_names=["input_size", "output_size", "trainable"])
```

| Layer (type, parameters)    | Input shape       | Output shape       | Trainable |
|-----------------------------|-------------------|--------------------|-----------|
| EfficientNet                | [32, 3, 288, 288] | [32, 3]            | Partial   |
| └ Sequential: 1-1           | [32, 3, 288, 288] | [32, 1408, 9, 9]   | False     |
| └ Conv2dNormActivation: 2-1 | [32, 3, 288, 288] | [32, 32, 144, 144] | False     |
| └ Conv2d: 3-1               | [32, 3, 288, 288] | [32, 32, 144, 144] | False     |

|                             |                    |                   |       |
|-----------------------------|--------------------|-------------------|-------|
| └─MBConv: 3-6               | [32, 16, 144, 144] | [32, 24, 72, 72]  | False |
| └─MBConv: 3-7               | [32, 24, 72, 72]   | [32, 24, 72, 72]  | False |
| └─MBConv: 3-8               | [32, 24, 72, 72]   | [32, 24, 72, 72]  | False |
| └─Sequential: 2-4           | [32, 24, 72, 72]   | [32, 48, 36, 36]  | False |
| └─MBConv: 3-9               | [32, 24, 72, 72]   | [32, 48, 36, 36]  | False |
| └─MBConv: 3-10              | [32, 48, 36, 36]   | [32, 48, 36, 36]  | False |
| └─MBConv: 3-11              | [32, 48, 36, 36]   | [32, 48, 36, 36]  | False |
| └─Sequential: 2-5           | [32, 48, 36, 36]   | [32, 88, 18, 18]  | False |
| └─MBConv: 3-12              | [32, 48, 36, 36]   | [32, 88, 18, 18]  | False |
| └─MBConv: 3-13              | [32, 88, 18, 18]   | [32, 88, 18, 18]  | False |
| └─MBConv: 3-14              | [32, 88, 18, 18]   | [32, 88, 18, 18]  | False |
| └─MBConv: 3-15              | [32, 88, 18, 18]   | [32, 88, 18, 18]  | False |
| └─Sequential: 2-6           | [32, 88, 18, 18]   | [32, 120, 18, 18] | False |
| └─MBConv: 3-16              | [32, 88, 18, 18]   | [32, 120, 18, 18] | False |
| └─MBConv: 3-17              | [32, 120, 18, 18]  | [32, 120, 18, 18] | False |
| └─MBConv: 3-18              | [32, 120, 18, 18]  | [32, 120, 18, 18] | False |
| └─MBConv: 3-19              | [32, 120, 18, 18]  | [32, 120, 18, 18] | False |
| └─Sequential: 2-7           | [32, 120, 18, 18]  | [32, 208, 9, 9]   | False |
| └─MBConv: 3-20              | [32, 120, 18, 18]  | [32, 208, 9, 9]   | False |
| └─MBConv: 3-21              | [32, 208, 9, 9]    | [32, 208, 9, 9]   | False |
| └─MBConv: 3-22              | [32, 208, 9, 9]    | [32, 208, 9, 9]   | False |
| └─MBConv: 3-23              | [32, 208, 9, 9]    | [32, 208, 9, 9]   | False |
| └─MBConv: 3-24              | [32, 208, 9, 9]    | [32, 208, 9, 9]   | False |
| └─Sequential: 2-8           | [32, 208, 9, 9]    | [32, 352, 9, 9]   | False |
| └─MBConv: 3-25              | [32, 208, 9, 9]    | [32, 352, 9, 9]   | False |
| └─MBConv: 3-26              | [32, 352, 9, 9]    | [32, 352, 9, 9]   | False |
| └─Conv2dNormActivation: 2-9 | [32, 352, 9, 9]    | [32, 1408, 9, 9]  | False |
| └─Conv2d: 3-27              | [32, 352, 9, 9]    | [32, 1408, 9, 9]  | False |
| └─BatchNorm2d: 3-28         | [32, 1408, 9, 9]   | [32, 1408, 9, 9]  | False |
| └─SiLU: 3-29                | [32, 1408, 9, 9]   | [32, 1408, 9, 9]  | --    |
| └─AdaptiveAvgPool2d: 1-2    | [32, 1408, 9, 9]   | [32, 1408, 1, 1]  | --    |
| └─Sequential: 1-3           | [32, 1408]         | [32, 3]           | True  |
| └─Dropout: 2-10             | [32, 1408]         | [32, 1408]        | --    |
| └─Linear: 2-11              | [32, 1408]         | [32, 3]           | True  |

```

Total params: 7,705,221
Trainable params: 4,227
Non-trainable params: 7,700,994
Total mult-adds (Units.GIGABYTES): 34.76

```

```

Input size (MB): 31.85
Forward/backward pass size (MB): 8291.71
Params size (MB): 30.82
Estimated Total Size (MB): 8354.38

```

```

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model_3.parameters(), lr=0.001)

# Set the random seeds
torch.manual_seed(42)
torch.cuda.manual_seed(42)

# Start the timer
from timeit import default_timer as timer
start_time = timer()

# Setup training and save the results
model_3_results = engine.train(model=model_3,
                                train_dataloader=train_dataloader_effnet_b2,
                                test_dataloader=test_dataloader_effnet_b2,
                                optimizer=optimizer,
                                loss_fn=loss_fn,
                                epochs=5,
                                device=device)

# End the timer and print out how long it took
end_time = timer()
print(f"[INFO] Total training time: {end_time-start_time:.3f} seconds")

```

```

100%          5/5 [00:12<00:00, 2.43s/it]

Epoch: 1 | train_loss: 1.1112 | train_acc: 0.3242 | test_loss: 0.9117 | test_acc: 0.7027
Epoch: 2 | train_loss: 0.8985 | train_acc: 0.6719 | test_loss: 0.8660 | test_acc: 0.8352
Epoch: 3 | train_loss: 0.8051 | train_acc: 0.8008 | test_loss: 0.7908 | test_acc: 0.8456
Epoch: 4 | train_loss: 0.7409 | train_acc: 0.7695 | test_loss: 0.6717 | test_acc: 0.9280
Epoch: 5 | train_loss: 0.6484 | train_acc: 0.8047 | test_loss: 0.6559 | test_acc: 0.8769
[INFO] Total training time: 12.108 seconds

```

Which one did the best?